

Normalization

Functional-Dependency Theory

Functional Dependencies

- There are usually a variety of constraints (rules) on the data in the real world.
- For example, some of the constraints that are expected to hold in a university database are:
 - Students and instructors are uniquely identified by their ID.
 - Each student and instructor has only one name.
 - Each instructor and student is (primarily) associated with only one department.
 - Each department has only one value for its budget, and only one associated building.

Functional Dependencies (Cont.)

- An instance of a relation that satisfies all such real-world constraints is called a **legal instance** of the relation;
- A legal instance of a database is one where all the relation instances are legal instances
- Constraints on the set of legal relations.
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes.
- A functional dependency is a generalization of the notion of a *key*.

Functional Dependencies Definition

- Let R be a relation schema

$$\alpha \subseteq R \text{ and } \beta \subseteq R$$

- The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β . That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A, B)$ with the following instance of r .

1	4
1	5
3	7

- On this instance, $B \rightarrow A$ hold; $A \rightarrow B$ does **NOT** hold,

Trivial Functional Dependencies

- A functional dependency is **trivial** if it is satisfied by all instances of a relation
- Example:
 - $ID, name \rightarrow ID$
 - $name \rightarrow name$
- In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

Use of Functional Dependencies

- We use functional dependencies to:
 - To test relations to see if they are legal under a given set of functional dependencies.
 - If a relation r is legal under a set F of functional dependencies, we say that r **satisfies** F .
 - To specify constraints on the set of legal relations
 - We say that F **holds on** R if all legal relations on R satisfy the set of functional dependencies F .
- Note: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances.
 - For example, a specific instance of *instructor* may, by chance, satisfy $name \rightarrow ID$.

Closure of a Set of Functional Dependencies

- Given a set F set of functional dependencies, there are certain other functional dependencies that are logically implied by F .
 - If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
 - etc.
- The set of **all** functional dependencies logically implied by F is the **closure** of F .
- We denote the *closure* of F by F^+ .

Closure of a Set of Functional Dependencies

- We can compute F^+ , the closure of F , by repeatedly applying **Armstrong's Axioms**:
 - **Reflexive rule**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
 - **Augmentation rule**: if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$
 - **Transitivity rule**: if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These rules are
 - **Sound** -- generate only functional dependencies that actually hold, and
 - **Complete** -- generate all functional dependencies that hold.

Example of F^+

- $R = (A, B, C, G, H, I)$
 $F = \{ A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H \}$
- Some members of F^+
 - $A \rightarrow H$
 - by transitivity from $A \rightarrow B$ and $B \rightarrow H$
 - $AG \rightarrow I$
 - by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$
and then transitivity with $CG \rightarrow I$
 - $CG \rightarrow HI$
 - by augmenting $CG \rightarrow I$ to infer $CG \rightarrow CGI$,
and augmenting of $CG \rightarrow H$ to infer $CGI \rightarrow HI$,
and then transitivity

Closure of Functional Dependencies (Cont.)

- Additional rules:
 - **Union rule**: If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
 - **Decomposition rule**: If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
 - **Pseudotransitivity rule**: If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.
- The above rules can be inferred from Armstrong's axioms.

Procedure for Computing F^+

- To compute the closure of a set of functional dependencies F :
 $F^+ = F$
repeat
 for each functional dependency f in F^+
 apply reflexivity and augmentation rules on f
 add the resulting functional dependencies to F^+
 for each pair of functional dependencies f_1 and f_2 in F^+
 if f_1 and f_2 can be combined using transitivity
 then add the resulting functional dependency to F^+
 until F^+ does not change any further
- **NOTE**: We shall see an alternative procedure for this task later

Closure of Attribute Sets

- Given a set of attributes α , define the **closure** of α under F (denoted by α^+) as the set of attributes that are functionally determined by α under F
- Algorithm to compute α^+ , the closure of α under F

```
closure :=  $\alpha$ ;
while (changes to closure) do
  for each  $\beta \rightarrow \gamma$  in  $F$  do
    begin
      if  $\beta \subseteq \text{closure}$  then closure := closure  $\cup \gamma$ 
    end
```

Example of Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
 - result = AG
 - result = ABCG ($A \rightarrow C$ and $A \rightarrow B$)
 - result = ABCGH ($CG \rightarrow H$ and $CG \subseteq AGBC$)
 - result = ABCGHI ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
 - Is AG a super key?
 - Does $AG \rightarrow R?$ == Is $R \supseteq (AG)^+$
 - Is any subset of AG a superkey?
 - Does $A \rightarrow R?$ == Is $R \supseteq (A)^+$
 - Does $G \rightarrow R?$ == Is $R \supseteq (G)^+$
 - In general: check for each subset of size $n-1$

Uses of Attribute Closure

There are several uses of the attribute closure algorithm:

- Testing for superkey:
 - To test if α is a superkey, we compute α^+ and check if α^+ contains all attributes of R .
- Testing functional dependencies
 - To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.
 - That is, we compute α^+ by using attribute closure, and then check if it contains β .
- Computing closure of F
 - For each $\gamma \subseteq R$, we find the closure γ^+ , and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.

Canonical Cover

- Suppose that we have a set of functional dependencies F on a relation schema. Whenever a user performs an update on the relation, the **database system must ensure that the update does not violate any functional dependencies**; that is, all the functional dependencies in F are satisfied in the new database state.
- If an update violates any functional dependencies in the set F , the system must roll back the update.
- We can reduce the effort spent in checking for violations by testing a simplified set of functional dependencies that has the same closure as the given set.
- This simplified set is termed the **canonical cover**
- To define canonical cover we must first define **extraneous attributes**.
 - An attribute of a functional dependency in F is **extraneous** if we can remove it without changing F^+

Extraneous Attributes

- An attribute of a functional dependency in F is **extraneous** if we can remove it without changing F^+ .
- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .
 - **Remove from the left side:** Attribute A is **extraneous** in α if
 - $A \in \alpha$ and
 - F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
 - **Remove from the right side:** Attribute A is **extraneous** in β if
 - $A \in \beta$ and
 - The set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F .
- *Note:* implication in the opposite direction is trivial in each of the cases above, since a “stronger” functional dependency always implies a weaker one

Extraneous Attributes

- **Removing an attribute from the left side of a functional dependency could make it a stronger constraint.**
 - For example, if we have $AB \rightarrow C$ and remove B , we get the possibly stronger result $A \rightarrow C$.
 - It may be stronger because $A \rightarrow C$ logically implies $AB \rightarrow C$, but $AB \rightarrow C$ does not, on its own, logically imply $A \rightarrow C$
- But, depending on what our set F of functional dependencies happens to be, **we may be able to remove B from $AB \rightarrow C$ safely.**
 - For example, suppose that
 - $F = \{AB \rightarrow C, A \rightarrow D, D \rightarrow C\}$
 - Then we can show that F logically implies $A \rightarrow C$, making extraneous in $AB \rightarrow C$.

Extraneous Attributes (Cont.)

- **Removing an attribute from the right side of a functional dependency could make it a weaker constraint.**
 - Suppose, if we have $AB \rightarrow CD$ and remove C , we get the possibly weaker result $AB \rightarrow D$. It may be weaker because using just $AB \rightarrow D$, we can no longer infer $AB \rightarrow C$.
- **But, depending on what our set F of functional dependencies happens to be, we may be able to remove C from $AB \rightarrow CD$ safely.**
 - For example, suppose that $F = \{AB \rightarrow CD, A \rightarrow C\}$.
 - Then we can show that even after replacing $AB \rightarrow CD$ by $AB \rightarrow D$, we can still infer $AB \rightarrow C$ and thus $AB \rightarrow CD$.

Testing if an Attribute is Extraneous

- Let R be a relation schema and let F be a set of functional dependencies that hold on R . Consider an attribute in the functional dependency $\alpha \rightarrow \beta$.
- **To test if attribute $A \in \beta$ is extraneous in β**
 - Consider the set: $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$,
 - check that α^+ contains A ; if it does, A is extraneous in β ,
- **To test if attribute $A \in \alpha$ is extraneous in α**
 - Let $\gamma = \alpha - \{A\}$. Check if $\gamma \rightarrow \beta$ can be inferred from F .
 - Compute γ^+ using the dependencies in F
 - If γ^+ includes all attributes in β then, A is extraneous in α

Examples of Extraneous Attributes

- Let $F = \{AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$
- To check if C is extraneous in $AB \rightarrow CD$, we:
 - Compute the attribute closure of AB under $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$
 - The closure is $ABCDE$, which includes CD
 - This implies that C is extraneous

Canonical Cover

A **canonical cover** for F is a set of dependencies F_c such that

- F logically implies all dependencies in F_c , and
- F_c logically implies all dependencies in F , and
- No functional dependency in F_c contains an extraneous attribute, and
- Each left side of functional dependency in F_c is unique. That is, there are no two dependencies in F_c
 - $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ such that
 - $\alpha_1 = \alpha_2$

Canonical Cover

- To compute a canonical cover for F :
 - repeat**
 - Use the union rule to replace any dependencies in F of the form $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$
 - Find a functional dependency $\alpha \rightarrow \beta$ in F_c with an extraneous attribute either in α or in β
 - /* Note: test for extraneous attributes done using F_c , not F */*
 - If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$
 - until** (F_c not change)
- Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied

Example: Computing a Canonical Cover

- $R = (A, B, C)$
 $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 - Set is now $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- A is extraneous in $AB \rightarrow C$
 - Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies
 - Yes: in fact, $B \rightarrow C$ is already present!
 - Set is now $\{A \rightarrow BC, B \rightarrow C\}$
- C is extraneous in $A \rightarrow BC$
 - Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies
 - Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
 - Can use attribute closure of A in more complex cases
- The canonical cover is:
 - $A \rightarrow B$
 - $B \rightarrow C$

Attribute Closure

- Suppose, a relational schema $R(A, B, C, D, E, F)$ and set of functional dependencies: $F \{ AB \rightarrow C, BC \rightarrow AD, D \rightarrow E, CF \rightarrow B \}$ Compute BCF^+, CD^+, D^+ .
- Suppose, a relational schema $R(A, B, C, D, E)$ and set of functional dependencies: $F \{ A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A \}$ Compute CD^+, E^+ .
- Suppose, a relational schema $R(A, B, C, D, E, F, G, H)$ and set of functional dependencies: $F \{ A \rightarrow BC, E \rightarrow C, AH \rightarrow D, CD \rightarrow E, D \rightarrow AEH, DH \rightarrow BC \}$ Compute AE^+ . Is $BCD \rightarrow H$ valid or not?

Computing a Canonical Cover

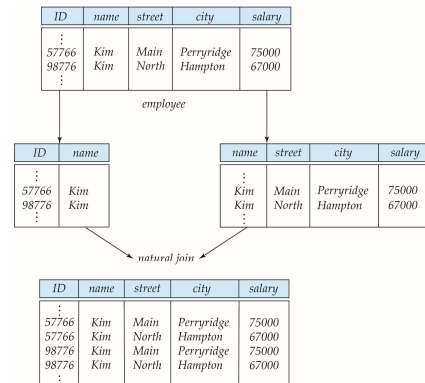
- Find the canonical cover of $F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$
Also find the candidate key/s.
- For the following set of FDs, find the minimal superkey and its irreducible set of FD
 $F = (A \rightarrow B, AB \rightarrow C, A \rightarrow BC, B \rightarrow C)$
Solution : $F = (A \rightarrow B, B \rightarrow C)$
- Suppose a relational schema $R(P, Q, R, S)$, and set of functional dependency as following $F : \{ P \rightarrow QR, Q \rightarrow R, P \rightarrow Q, PQ \rightarrow R \}$
Find the canonical cover F_c
Solution: $F_c = \{ P \rightarrow Q, Q \rightarrow R \}$
- For a relational schema $R(A, B, C, D)$ and a set of FDs $F = (B \rightarrow A, AD \rightarrow BC, C \rightarrow ABD)$, find F_c
Solution : $F_c (B \rightarrow A, AD \rightarrow C, C \rightarrow BD)$

Decomposition

- The only way to avoid the repetition-of-information problem in the *in_dep* schema is to decompose it into two schemas – *instructor* and *department* schemas.
- Not all decompositions are good. Suppose we decompose
 $employee(ID, name, street, city, salary)$
into
 $employee1(ID, name)$
 $employee2(name, street, city, salary)$

The problem arises when we have two employees with the same name
- The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**.

A Lossy Decomposition



Lossless Decomposition

- Let R be a relation schema and let R_1 and R_2 form a decomposition of R . That is $R = R_1 \cup R_2$
- We say that the decomposition is a **lossless decomposition** if there is no loss of information by replacing R with the two relation schemas $R_1 \cup R_2$
- Formally,

$$\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$
- And, conversely a decomposition is lossy if

$$r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

Example of Lossless Decomposition

- Decomposition of $R = (A, B, C)$
 $R_1 = (A, B)$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B

Lossless Decomposition

- We can use functional dependencies to show when certain decomposition are lossless.
- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$
- A decomposition of R into R_1 and R_2 is lossless decomposition if at least one of the following dependencies is in F^+ :
 - $R_1 \cap R_2 \rightarrow R_1$
 - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies

Example

- $R = (A, B, C)$
 $F = \{A \rightarrow B, B \rightarrow C\}$
- $R_1 = (A, B)$, $R_2 = (B, C)$
 - Lossless decomposition:

$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow BC$$
- $R_1 = (A, B)$, $R_2 = (A, C)$
 - Lossless decomposition:

$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$
- Note:
 - $B \rightarrow BC$
 is a shorthand notation for
 $B \rightarrow \{B, C\}$

Dependency Preservation

- Testing functional dependency constraints each time the database is updated can be costly
- It is useful to design the database in a way that constraints can be tested efficiently.
- If testing a functional dependency can be done by considering just one relation, then the cost of testing this constraint is low
- While decomposing a relation it is no longer possible to do the testing without having to perform a Cartesian Product.
- A decomposition that makes it computationally hard to enforce functional dependency is said to be NOT **dependency preserving**.

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i .
 - A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$
- Using the above definition, testing for dependency preservation take exponential time.
- Not that if a decomposition is NOT dependency preserving then checking updates for violation of functional dependencies may require computing joins, which is expensive.

Dependency Preservation (Cont.)

- Let F be the set of dependencies on schema R and let $R_1, R_2, \dots, R_n$ be a decomposition of R .
- The restriction of F to R_i is the set F_i of all functional dependencies in F^+ that include **only** attributes of R_i .
- Since all functional dependencies in a restriction involve attributes of only one relation schema, it is possible to test such a dependency for satisfaction by checking only one relation.
- Note that the definition of restriction uses all dependencies in F^+ , not just those in F .
- The set of restrictions $F_1, F_2, \dots, F_n$ is the set of functional dependencies that can be checked efficiently.

Testing for Dependency Preservation

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \dots, R_n$, we apply the following test (with attribute closure done with respect to F)
 - $result = \alpha$
 - **repeat**
 - for each** R_i in the decomposition

$$t = (result \cap R_i)^+ \cap R_i$$

$$result = result \cup t$$
 - until** ($result$ does not change)
 - If $result$ contains all attributes in β , then the functional dependency $\alpha \rightarrow \beta$ is preserved.
- We apply the test on all dependencies in F to check if a decomposition is dependency preserving
- This procedure takes polynomial time, instead of the exponential time required to compute F^+ and $(F_1 \cup F_2 \cup \dots \cup F_n)^+$

Dependency Preservation Example

- Consider a schema:
dept_advisor(s_ID, i_ID, department_name)
- With function dependencies:
 $i_ID \rightarrow dept_name$
 $s_ID, dept_name \rightarrow i_ID$
- In the above design we are forced to repeat the department name once for each time an instructor participates in a *dept_advisor* relationship.
- To fix this, we need to decompose *dept_advisor*
- Any decomposition will not include all the attributes in
 $s_ID, dept_name \rightarrow i_ID$
- Thus, the composition NOT be dependency preserving

Normal Forms

- First Normal Form:** A relvar is in 1NF if and only if, in every legal value of that relvar, every tuple contains exactly one value for each attribute.

Consider a relvar **Supplier_Ship (S#,P#)** with $City \rightarrow Status$

<u>S#</u>	Status	City	<u>P#</u>	Qty
S1	20	Delhi	P1	300
S1	20	Delhi	P2	200
S2	10	Mumbai	P3	250
S3	15	Chennai	P2	300
S4	20	Delhi	P1	400
S4	20	Delhi	P4	240
S5	10	Mumbai	P4	310

Update Anomalies:

- Insert :** We cannot insert a particular supplier located in a city until that supplier supplies atleast one part.
- Delete :** If we delete the supplier S3, we delete not only the shipment details of the supplier but also deletes the information that supplier S3 was located in the Chennai.
- Update :** The city value for a given supplier appears many times. Updating the city value of a supplier requires to update the city value in all the tuples of that particular supplier.

- Second Normal Form :** A relvar is in second normal form if and only if it is 1NF and every nonkey attribute is irreducibly dependent on the primary key.
- Supplier-Ship (S#, P#, City, Status, Qty)** with $City \rightarrow Status, S\# \rightarrow City$ and $S\# \rightarrow Status$
- The relvar Supplier-Ship is not in 2NF, so to bring it into 2NF we decompose it into two relvars: **Supplier (S#, City, Status)** and **Shipment (S#, P#, Qty)**

Supplier (S#, City, Status)

<u>S#</u>	Status	City
S1	20	Delhi
S2	10	Mumbai
S3	15	Chennai
S4	20	Delhi
S5	10	Mumbai

Shipment (S#, P#, Qty)

<u>S#</u>	<u>P#</u>	Qty
S1	P1	300
S1	P2	200
S2	P3	250
S3	P2	300
S4	P1	400
S4	P4	240
S5	P4	310

Update Anomalies:

- **Insert** : We cannot insert the fact that a particular city has a particular status e.g. cannot say that city Bangalore has a status 50 until we have a supplier from that city
- **Delete** : If we delete the sole tuple supplier S3, we delete not only the information for the supplier concerned but also the information that the city has a particular status.
- **Update** : The status for a given city appears many times, and if we need to change the status of a city Delhi from 20 to 30, we are faced with either the problem of searching the relvar to find every tuple for Delhi and then changing the status or we would produce the inconsistent result.

- **Third Normal Form** : A relvar is in third normal form if and only if it is 2NF and every non-key attribute is non-transitively dependent on the primary key.
- **Supplier (S#, City, Status)** with $\text{City} \rightarrow \text{Status}$, $\text{S#} \rightarrow \text{City}$ and $\text{S#} \rightarrow \text{Status}$
- The relvar Supplier is not in 3NF, so to bring it into 3NF we decompose it into two relvars: **SC (S#, City)** and **CS (City, Status)**

Supplier (S#, City) CityStatus (City, Status) Shipment (S#, P#, SName Qty)

<u>S#</u>	City
S1	Delhi
S2	Mumbai
S3	Chennai
S4	Delhi
S5	Mumbai

City	Status
Delhi	20
Mumbai	10
Chennai	15

<u>S#</u>	<u>P#</u>	SName	Qty
S1	P1	A	300
S1	P2	A	200
S2	P3	B	250
S3	P2	C	300
S4	P1	D	400
S4	P4	D	240
S5	P4	E	310

- Let us assume supplier names to have unique names.
- A relvar Supplier has **two candidate keys** i.e. **S# and Sname**
- These keys are disjoint candidate keys i.e. no common attribute.
- So, FDs in Supplier
 - $S\# \rightarrow City$, $Sname \rightarrow City$,
 - $S\# \rightarrow Sname$ and $Sname \rightarrow S\#$
- Question 1: Does Supplier relvar is in 2NF ??**
- Question 2: Does Supplier relvar is in 3NF ??**

Supplier (S#, Sname, City)

S#	Sname	City
S1	A	Delhi
S2	B	Mumbai
S3	C	Chennai
S4	D	Delhi
S5	E	Mumbai

- Let us assume supplier names to have unique names.
- In this case, we have two candidate keys for relvar Shipment i.e.
 - Candidate Key 1: **S#, P#**
 - Candidate Key 2: **Sname, P#** (as Supplier names are unique)
- The relvar Shipment has multiple i.e. two candidate keys
- These keys are **composite candidate keys**
- These candidate keys are **overlapping keys** i.e. **they share a common attribute**
- So FDs in Shipment
 - $S\#, P\# \rightarrow Qty$
 - $Sname, P\# \rightarrow Qty$
 - Any other FD /s ??

Shipment (S#, P#, Sname Qty)

S#	P#	Sname	Qty
S1	P1	A	300
S1	P2	A	200
S2	P3	B	250
S3	P2	C	300
S4	P1	D	400
S4	P4	D	240
S5	P4	E	310

- Question 1: Does relvar Shipment in 2NF ??**
- Question 2: Does relvar Shipment in 3NF ??**

Boyce/Codd Normal Form (BCNF)

- A relvar is in BCNF if and only if every non-trivial, left irreducible FD has a candidate key as its determinant
- OR
- A relvar is in BCNF if and only if every determinant is a candidate key

Normalization (contd...)

- In relation CTR, the attributes corresponding to teachers and texts are relation-valued i.e. CTR – CT and CTR – CR.
- For any course, there can be any number of teachers who can teach the course.
- For any course there can be any number of corresponding reference texts/ books
- In the given relation:
 - a tuple {Course: c, Teacher: t, Text: x} appears in CTR if and only if
 - Course c can be taught by Teacher and uses a text x as a reference text
 - CTR satisfies the constraint:
 - If tuples {c, t1, x1}, {c, t2, x2} both appears
 - Then tuples {c, t1, x2}, {c, t2, x1} should also appear

CTR

Course	Teacher	Reference Text
DS & Algo	Prof. Vijay	Coreman
	Prof. Satish	Tanenbaum Sartaj Sahni
DBMS	Dr Vikas	Korth Navathe Desai

Normalization (contd...)

Course	Teacher	Reference Text
DS & Algo	Prof. Vijay	Cormen
	Prof. Satish	Tanenbaum
		Sartaj Sahni
DBMS	Dr Vikas	Korth
		Navathe
		Desai

If tuples {DS&Algo, Prof Vijay, Cormen} and {DS&Algo, Prof Satish, Tanenbaum} appear
Then tuples {DS&Algo, Prof Vijay, Tanenbaum} and {DS&Algo, Prof Satish, Cormen} Should also appear

Course	Teacher	Reference Text
DS & Algo	Prof Vijay	Cormen
DS & Algo	Prof Vijay	Tanenbaum
DS & Algo	Prof Vijay	Sartaj Sahni
DS & Algo	Prof Satish	Cormen
DS & Algo	Prof Satish	Tanenbaum
DS & Algo	Prof Satish	Sartaj Sahni
DBMS	Dr Vikas	Korth
DBMS	Dr Vikas	Navathe
DBMS	Dr Vikas	Desai

Normalization (contd..)

- Although Teachers and Texts are completely independent of each other
- but relvar constraint forces us to insert couple of tuples i.e. {DS&Algo, Prof Vijay, Tanenbaum} and {DS&Algo, Prof Satish, Cormen}
- If tuples {DS&Algo, Prof Vijay, Cormen} and {DS&Algo, Prof Satish, Tanenbaum} exists.
- This is because of multi-valued dependency of Teachers and Reference Texts on Course
- A multi-valued dependency (MVD) $A \twoheadrightarrow B$ holds on a relation R, if for a particular value of attribute A, the values for attribute B is independent of the values of attributes of R other than A and B.
- In the relvar CTR, for a given course c and a given text x, the set of teachers t matching the pair (c, x) in CTR depends on the value of c alone and makes no difference which particular value of x we choose.
- In relvar CTR, Course $\twoheadrightarrow$ Teacher and Course $\twoheadrightarrow$ Text

Fourth Normal Form (4NF)

- Multi-valued dependencies are generalization of functional dependency.
- Every FD is an MVD, but the converse is not true.

Course $\twoheadrightarrow$ Teacher and Course $\twoheadrightarrow$ Text

Though a course doesn't have a single corresponding Teacher i.e. functional dependency Course $\rightarrow$ Teacher doesn't hold.

Fourth Normal Form

- A relation is in 4NF, if for every non-trivial multi-valued dependency (MVD) $A \twoheadrightarrow B$ in R, A is a superkey

Join Dependency and 5NF

Join Dependency : Let R be a relation and X1, X2, Xk be the subset of attributes of R then

JD[X1, X2 Xk] holds on relation R if

$$R = \text{Proj}(X1) \bowtie \text{Proj}(X2) \bowtie \dots \text{Proj}(Xk)$$

A relation can be non-loss decomposed into 'n' projections but not into 'm' projections for any $m < n$

Trivial JD:- When one of the projection Xi's contain all attributes of R

5NF:- A relation R is in 5NF with respect to FD's, MVD's and JD's if for every non-trivial JD, JD[X1, X2, Xk] one of the Xj's is a super key

JoinDependency (contd..)

S#	P#	J#
S1	P1	J2
S1	P2	J1
S2	P1	J1
S1	P1	J1

S#	P#	J#
S1	P1	J2
S1	P2	J1
S2	P1	J1
S2	P1	J2
S1	P1	J1

S#	P#
S1	P1
S1	P2
S2	P1

P#	J#
P1	J2
P2	J1
P1	J1

J#	S#
J2	S1
J1	S1
J1	S2